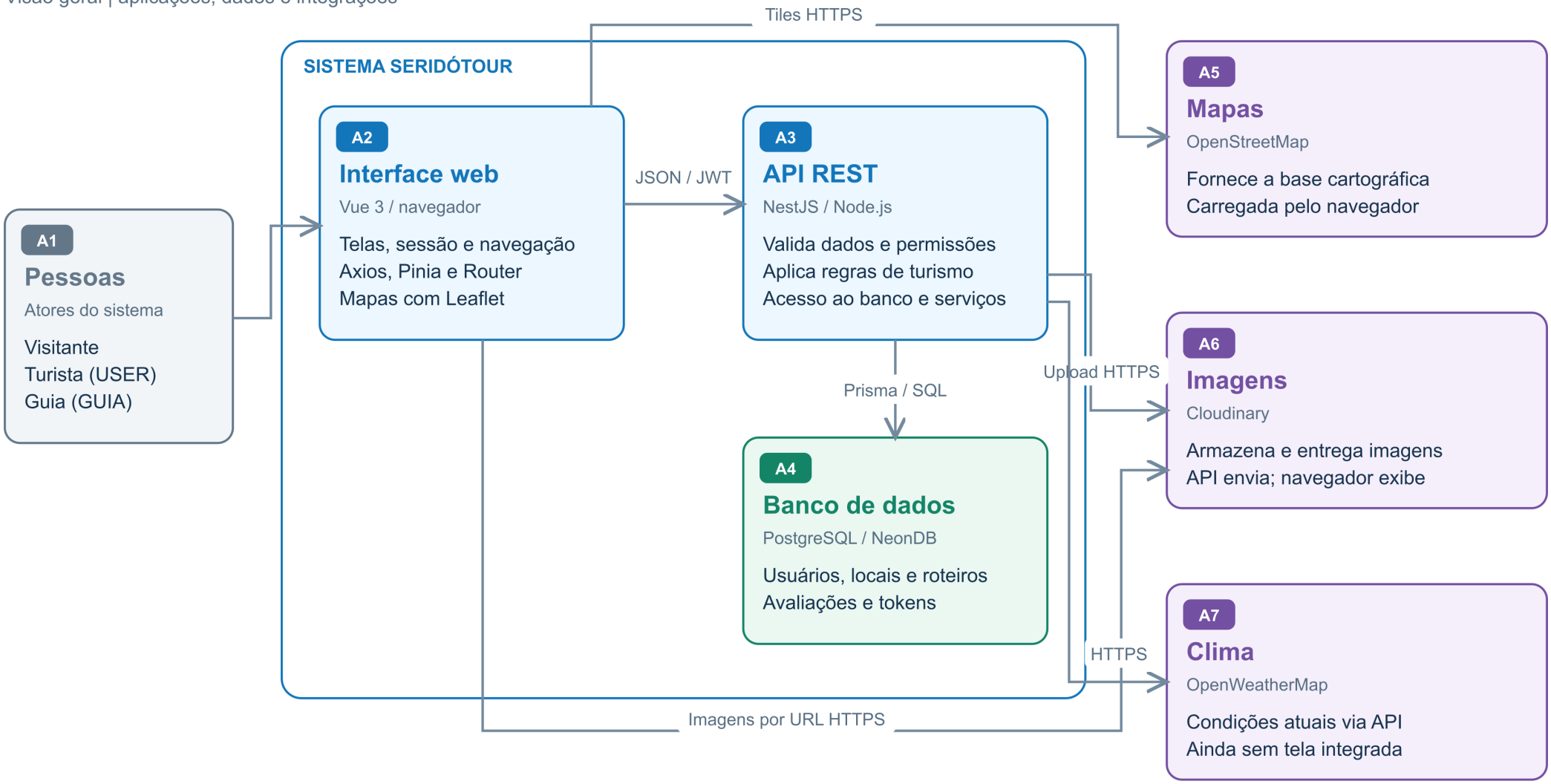
SERIDÓTOUR / PROJETO INTEGRADOR

Arquitetura do sistema

Visão geral | aplicações, dados e integrações



Azul: aplicação • Verde: dados • Roxo: serviço externo • Setas: chamadas; respostas retornam ao chamador.

O frontend executa no navegador. O Prisma pertence à API. O navegador não acessa o PostgreSQL.

Arquitetura do sistema

Aplicação cliente-servidor: interface, API, dados e serviços externos.

A1 - Pessoas

Visitante consulta informações públicas; turista cria roteiros e avaliações; guia também gerencia locais, consulta usuários e modera roteiros. No banco, os papéis são USER e GUIA.

A2 - Interface web

Vue 3 apresenta telas e formulários. Vue Router controla a navegação, Pinia mantém a sessão e Axios envia chamadas à API. Leaflet apresenta os mapas. O código da interface executa no navegador.

A3 - API REST

NestJS recebe as chamadas, valida dados e permissões e aplica regras de negócio. JWT identifica o usuário; o papel GUIA libera as operações administrativas.

A4 - Banco de dados

PostgreSQL, hospedado no NeonDB em produção, armazena usuários, tokens, locais, roteiros e avaliações. Prisma faz o acesso ao banco dentro da API. Imagens são armazenadas como URLs.

A5 - OpenStreetMap

Fornece as imagens da base do mapa diretamente ao navegador. Os marcadores usam coordenadas dos locais cadastrados. Não há cálculo automático de trajetos.

A6 - Cloudinary

Recebe imagens enviadas pelo backend. O banco guarda a URL retornada e o navegador usa essa URL para exibir a imagem.

A7 - OpenWeatherMap

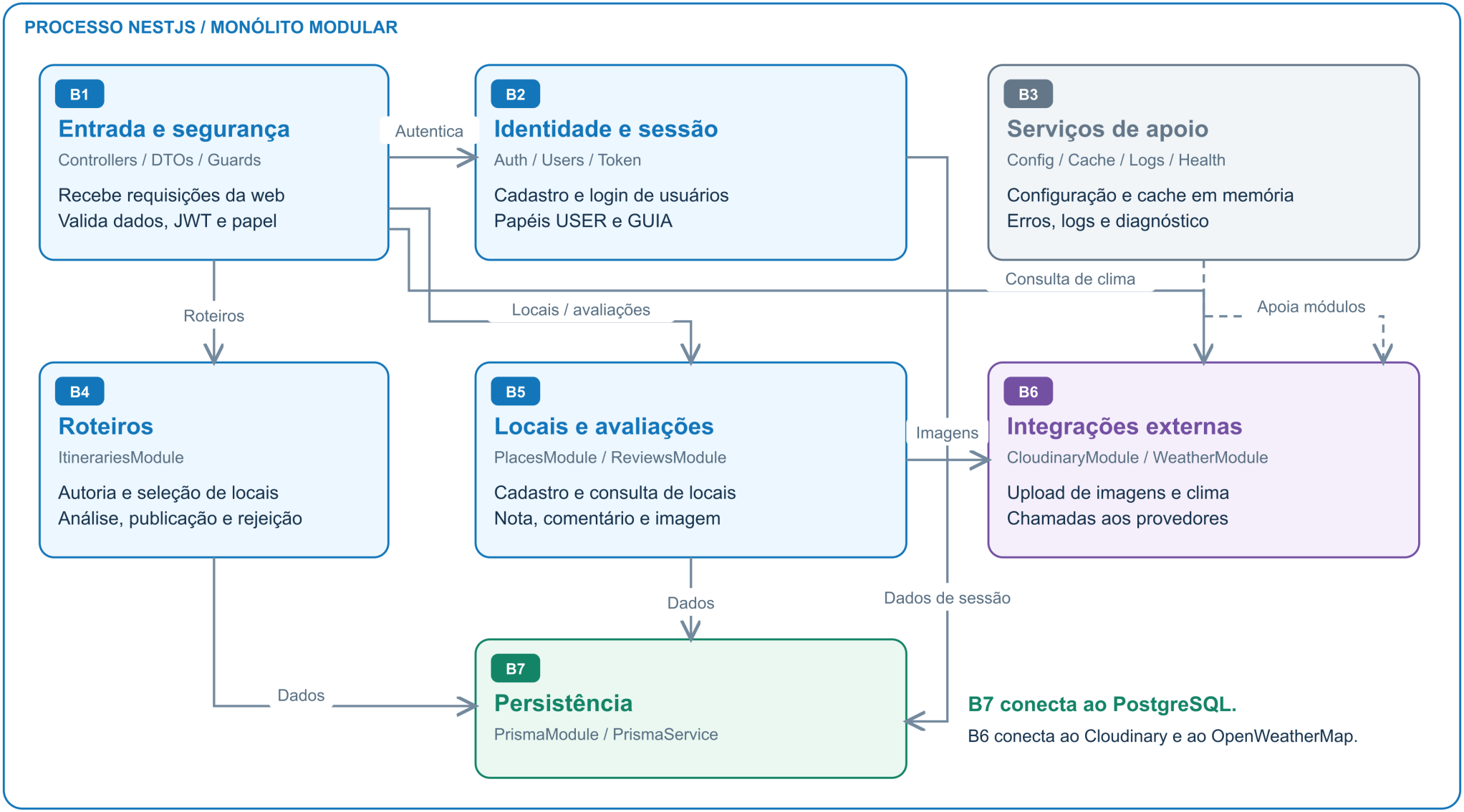
Fornece condições meteorológicas atuais, consultadas pelo backend. A função existe na API, mas a tela de roteiro ainda não a utiliza.

Fluxo principal

A interface envia uma ação à API com o token, quando necessário. A API valida a permissão, aplica as regras e usa o Prisma para persistir os dados. O resultado retorna à interface. No upload, o arquivo vai ao Cloudinary e a URL fica no banco.

Por dentro da API

Visão de componentes | um único backend NestJS, organizado em módulos



Blocos agrupam responsabilidades. Não representam microsserviços ou servidores separados.

Componentes da API

Os módulos dividem responsabilidades dentro de uma única aplicação NestJS.

B1 - Entrada e segurança

Controllers recebem as requisições. DTOs e validações conferem campos e arquivos. Guards verificam o token JWT e, quando necessário, o papel de guia.

B2 - Identidade e sessão

Auth, Users e Token cuidam do cadastro, senha protegida, login, papéis e registro de tokens. O gerenciamento de usuários disponível permite somente listar contas.

B3 - Serviços de apoio

Config carrega variáveis de ambiente; cache em memória reaproveita algumas consultas; logs e filtros registram e padronizam erros; Health expõe o diagnóstico em /health. Swagger documenta a API em /api-docs quando habilitado.

B4 - Roteiros

Itineraries controla autor, título, duração, locais e situação. Um roteiro em análise não pode ser editado. O autor envia para análise e o guia publica, rejeita ou despublica.

B5 - Locais e avaliações

Places mantém os pontos turísticos. Reviews registra nota, comentário, autor, local e imagem opcional. O comentário é parte da avaliação.

B6 - Integrações externas

CloudinaryModule encapsula o upload de imagens. WeatherModule consulta OpenWeatherMap e trata falhas do provedor. Mapas não passam por esse módulo: o navegador consulta OpenStreetMap.

B7 - Persistência

PrismaService é usado pelos módulos que consultam ou alteram dados. O schema Prisma define entidades e relacionamentos no PostgreSQL. Não é um servidor separado.

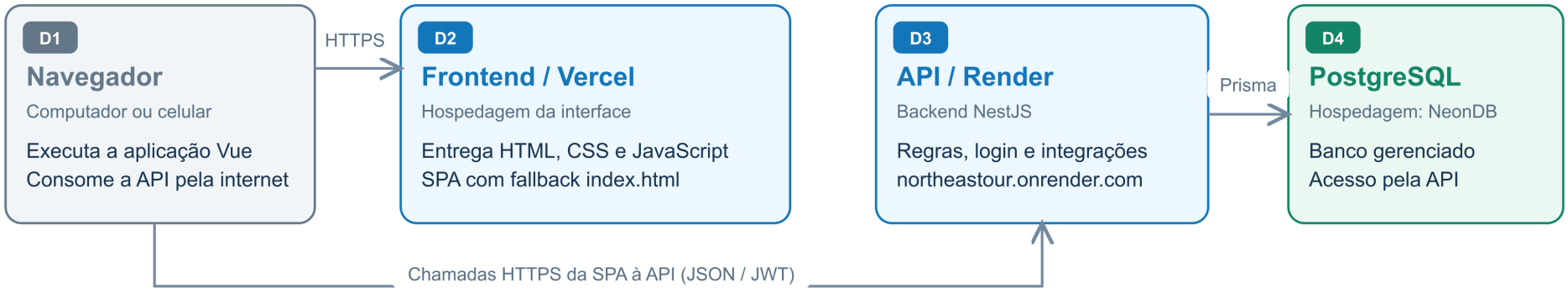
Organização

O monorepo usa npm workspaces. Controllers recebem chamadas; services executam regras; o Prisma concentra a persistência. O cache é local à API. Nenhum bloco interno representa um servidor independente.

Arquitetura de deploy

Ambiente publicado + ambiente local | verificação em 09/11/2026

AMBIENTE PUBLICADO



Frontend: northeas-tour-frontend.vercel.app

Vercel e URL Render identificadas no site público; API respondeu 200 em consulta de locais.

AMBIENTE LOCAL / DOCKER COMPOSE



Cloudinary, OpenStreetMap e OpenWeatherMap continuam externos, conforme a visão geral.

Docker Compose usa alvos dev. Não foram encontrados pipeline CI/CD nem configuração versionada do deploy Render.

Deploy e execução local

Ambientes publicados e locais possuem formas diferentes de execução.

D1 - Navegador

Baixa a interface e executa o Vue. Depois faz chamadas HTTPS diretamente à API, enviando o token nas operações protegidas.

D2 - Vercel

Hospeda o frontend em northeas-tour-frontend.vercel.app. A configuração vercel.json redireciona rotas da SPA para index.html.

D3 - Render

Hospeda a API configurada no frontend em northeastour.onrender.com. O backend atende chamadas, aplica regras e consulta banco e integrações.

D4 - PostgreSQL / NeonDB

Banco PostgreSQL hospedado no NeonDB. A API no Render acessa o banco por DATABASE_URL, utilizando o Prisma.

L1 - Frontend

Container com Vite, disponível por padrão em localhost:5173. VITE_API_URL aponta para localhost:3000 porque a chamada à API parte do navegador no computador do usuário.

L2 - Backend

Container NestJS, publicado em localhost:3000. Acessa o banco pelo nome postgres e porta interna 5432. Na inicialização atual gera o cliente Prisma, sincroniza o schema com db push e inicia o modo de desenvolvimento.

L3 - PostgreSQL

Container postgres:15-alpine. Publica a porta 5433 do computador para 5432 do container; o volume postgres_data mantém os dados.

Deploy observado

Vercel e URL Render identificadas em 09/11/2026 no site público. A API respondeu HTTP 200 em GET /places?limit=1. O PostgreSQL é hospedado no NeonDB, conforme informado pelo responsável pelo projeto. O Docker Compose usa alvos dev; não foi encontrado pipeline CI/CD versionado.

Configuração e fontes

Como os blocos se conectam em cada ambiente.

Endereços publicados

Frontend: https://northeas-tour-frontend.vercel.app
API: https://northeastour.onrender.com
Evidência registrada em docs/diagramas/deploy-verificado.json.

Variáveis da comunicação

VITE_API_URL define o destino das chamadas do frontend no build. CORS_ORIGIN permite a origem do frontend no backend. DATABASE_URL conecta a API ao PostgreSQL no NeonDB em produção e ao serviço postgres no Docker local. JWT_SECRET e variáveis de expiração configuram a autenticação. CLOUDINARY_* e OPENWEATHER_API_KEY ficam no ambiente do backend.

Docker local e produção

Localmente, o navegador usa localhost:5173 para a interface e localhost:3000 para a API. A API usa postgres:5432 dentro da rede Docker. Os Dockerfiles também possuem alvos prod: Nginx para a interface e Node.js para a API; não foi confirmado que o Render os utiliza.

Limites da versão

A tela de roteiro ainda não usa o módulo de clima. Gerenciar usuários permite somente listar contas. A documentação representa o código analisado; o commit implantado na nuvem não foi identificado.

Fontes do projeto

github.com/SeridoTour/SeridoTour
Base local main: 0dd906a31a582a196f68c03534e222574416b1d7.
Arquivos: docker-compose.yml; Dockerfiles; vercel.json; AppModule; main.ts; cliente Axios; schema.prisma.

Referência visual

Modelo C4: c4model.com/diagrams/container e c4model.com/diagrams/deployment. Os desenhos são próprios e usam notação livre de blocos, permitida pelo enunciado.